Brianna Stewart
CSCI-375-01
February, 2026

Code Review Report

The assignment tasked us with implementing a Boggle game solver capable of finding all valid words on a board given a dictionary of allowed words. My approach leveraged a Trie (prefix tree) for efficient storage and lookup of dictionary words, which allows rapid checking of both complete words and valid prefixes. The solver performs a depth-first search (DFS) with backtracking to explore all potential word paths across the grid. Special handling was included for the Boggle "Q" tile by automatically converting "Q" into "QU" during traversal. The solver enforces a configurable minimum word length, defaulting to three, and validates that the board is rectangular to prevent runtime errors.

During the peer review process, AndresHU2028 provided several key points. First, he noted that while using a prefix set effectively prunes DFS paths, storing all possible prefixes consumes considerable memory. He recommended using a Trie structure to store prefixes in a shared path, reducing memory usage. He also pointed out that in the existing code, the prefix set is not rebuilt when the dictionary changes, which could lead to incorrect behavior if new words are introduced. Additionally, he suggested ensuring that words starting with "QU" are handled properly, as Boggle rules treat "Q" as "QU" in all cases. AndresHU2028 also confirmed that the backtracking and DFS logic are well implemented, particularly the recursive exploration of neighbors and the marking and unmarking of visited tiles. He highlighted that the current minimum word length is hardcoded as 3, which could limit flexibility if the rules change. Finally, he emphasized the importance of validating the grid to ensure it is rectangular, which prevents potential IndexErrors during traversal.

In response, I recognized that the solver effectively uses DFS and prefix pruning to efficiently search the board, and I highlighted the performance advantage of stopping paths that cannot form valid words. I also noted that the code assumes a square board; supporting rectangular boards by storing separate row and column counts would improve flexibility. Additionally, I emphasized the importance of normalizing grid letters to uppercase to maintain consistency during comparisons and recommended replacing the hardcoded minimum word length with a constant or variable.

Based on the review, I implemented several improvements. The prefix set was replaced with a Trie-based structure, reducing memory usage and maintaining efficient prefix checks. The “build_trie” method now ensures that the Trie is rebuilt whenever the dictionary changes, preventing stale prefix data. The constructor now accepts a minimum word parameter, allowing the minimum valid word length to be customized instead of hardcoding it. Rectangular grid validation was added to confirm that all rows have consistent lengths, preventing runtime errors in non-square boards. Handling of the "Q" tile was improved through the gmethod, which

automatically converts any "Q" tile to "QU". Additionally, the code was cleaned up for PEP8 compliance, including consistent naming, spacing, proper indentation, docstrings, and adding a final newline to eliminate the W292 warning. These changes strengthened the overall maintainability, readability, and flexibility of the solver.

Static analysis confirmed that all previously reported style and linting issues were resolved. The addition of the final newline fixed W292 warnings, and variable and function naming were aligned with PEP8 standards. The code was thoroughly regression-tested to verify correctness. All example grids returned the expected words, rectangular boards such as 3 by 5 and 4 by 3 were correctly processed, and the `min_word_length` parameter functioned as expected. Tests also confirmed that "QU" tiles are properly handled, and the outputs remained consistent following the Trie refactoring.

Reflecting on the review process, I found that peer feedback was invaluable in highlighting both efficiency considerations and edge cases that could have been overlooked. The process emphasized the importance of flexibility, such as supporting rectangular boards and configurable minimum word lengths, and showed how data structure choices impact performance, as seen with the Trie replacing a flat prefix set. I also gained insight into professional workflows, including iterative review cycles, constructive feedback, and documenting both improvements and testing procedures. Overall, the review reinforced that well-structured, maintainable code is as important as algorithmic correctness. The final implementation demonstrates strong recursion and backtracking techniques, effective use of data structures for optimization, and clean, readable code. With these improvements, the Boggle solver is robust, flexible, and production-ready while still being fully compliant with assignment requirements.